MINAMIAI · CAPSTONE PROJECT

Frontend planning

Screens, navigation, and component structure for MinamiAI: an interactive chat window with real-time TTS and an animated avatar.

Control flow

The app has two entry points that never intersect during normal use. Ordinary users land on the **`Auth screen`**, and a successful login or registration takes them into the **`Chat scene`**, which is where nearly all product time is spent. Logging out returns to the Auth screen. The **`Admin panel`** sits outside this flow entirely — it is a separate static page an admin account reaches by URL rather than a screen the React app routes to, and it has its own password gate independent of the JWT used everywhere else.

From	Action	To
Auth screen	Submit valid login or registration	Chat scene
Auth screen	Invalid credentials or validation error	Auth screen (inline error)
Chat scene	Logout	Auth screen
—	Admin visits `/admin` directly	Admin panel

Screens

BUILT **`Auth screen`** · AuthPage.jsx

- Login and registration, toggled within a single view rather than separate routes.
- On success, stores the returned JWT and username and hands control up to the app root, which switches to the Chat scene.
- On failure, shows the error inline rather than redirecting, matching the behavior specified in the API planning document for `401` / `400` responses.

BUILT **`Chat scene`** · ChatPage.jsx

- Full-screen animated canvas background (radial gradients, hexagons, particles, scan line) with the Live2D model composited over it — no containing box around the character.
- A glassmorphism chat panel slides in from the right and can be toggled away to leave the character full-screen; message list, loading indicator, and input row live inside it.
- A header shows the logged-in username and a logout control.
- On mount, requests or resumes a `conversation_id` from the backend before the panel can send messages.
- Speech output triggers `isSpeaking`, which drives the avatar's lip sync; a request or TTS failure runs the app's error path (red-tinted error bubble, surprised expression) instead of failing silently.

STATIC **`Admin panel`** · admin.html, served at /admin

- Not part of the React app or its client-side state — a standalone HTML page gated by a separate admin password rather than the user JWT.
- Full CRUD over the memory store: filter by keyword, type, importance, and date; sort; bulk select, delete, or re-tier.
- A user switcher lets an admin inspect and edit any user's memories, which is what makes this a debugging tool for the memory system as much as an administrative feature.

Component structure — Chat scene

The Chat scene is the only screen with meaningful internal structure today. State currently lives in `ChatPage` and is passed down as props; nothing above it is shared through context.

Component	Owns	Notes

`ChatPage`	`messages`, `input`, `conversationId`, `loading`, `isSpeaking`, `audioRef`	Top-level state owner for the screen; fetches the conversation on mount, sends messages, and drives TTS playback.
`Avatar`	`canvasRef`, `modelRef`, `appRef` (internal)	Receives `isSpeaking` as its only prop; owns the PixiJS application, the Live2D model instance, and the mouse-tracking listeners that drive head and eye parameters.
Chat panel (message list, input row)	Renders from `ChatPage`'s state; no local state of its own beyond toggle-open/closed	Not yet split into separate component files — currently inline JSX within `ChatPage`.

State management

There is no global state library in the frontend — no Redux, no Context API beyond whatever React provides implicitly. Every screen manages its own state with `useState`, and cross-screen values (the JWT, the username) are passed down as props from the app root rather than read from a shared store. This is workable with two screens. It will not stay workable once the MinamiTask dashboard needs the same auth state the Chat scene already has, plus its own routine list, without either duplicating fetch logic or drilling props through an extra layer.

Known gaps

- **No client-side router.** Navigation between Auth and Chat is a conditional render at the app root based on whether a token exists, not `react-router`. That is fine for the two screens the app has today, but would need revisiting if the app ever grows a third in-app screen.
- **No shared auth context.** Token and username live wherever the app root put them and get threaded down as props. A Context provider wrapping the authenticated portion of the app would remove the need to keep passing them explicitly as the component tree grows.
- **Admin panel is architecturally separate.** It does not share a login, a build step, or a component with the React app. That has been fine as an internal tool, but it means any UI pattern built for the main app (the error-bubble treatment, the glassmorphism panel style) has to be reimplemented by hand if it is ever wanted there too.
- **Chat panel has no component boundaries yet.** Message list and input row are inline JSX in `ChatPage` rather than their own components, which makes the file harder to navigate as it grows.